MASARYK UNIVERSITY

FACULTY OF INFORMATICS

Implementation of MCP Server for Red Hat OpenShift AI Workbenches

Bachelor's Thesis

ADAM DAVID MALÝ

Brno, Fall 2025

MASARYK UNIVERSITY

FACULTY OF INFORMATICS

Implementation of MCP Server for Red Hat OpenShift AI Workbenches

Bachelor's Thesis

ADAM DAVID MALÝ

Advisor: Honza + Martin

?

Brno, Fall 2025

MUNI
FI

Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

Adam David Malý

Advisor: Honza + Martin

Abstract

This thesis presents MCP server implementation for RHOAI workbenches.

Keywords

MCP, RHOAI, Workbench, Server, Implementation

Contents

List of Tables

List of Figures

1 Introduction

Large language models (LLMs) have rapidly evolved from conversational assistants into capable agents that can reason, plan, and take actions in the real world. This shift has been made possible in part by the ability to give LLMs access to external *tools* — functions that let a model query a database, call an API, or manage infrastructure, all in response to a natural language request. As the ecosystem of such integrations grows, a key challenge emerges: how do different AI hosts and models connect to the ever-expanding set of tools and data sources in a consistent, interoperable way?

Anthropic introduced the Model Context Protocol (MCP) in November 2024 [1] to address exactly this problem. MCP is an open standard that defines how AI applications connect to external tools and data sources through a client-server architecture. Instead of each AI host maintaining its own bespoke integrations, any MCP-compatible host — such as Cursor, Claude Desktop, or a custom agent — can connect to any MCP server and immediately gain access to the capabilities it exposes. This has already led to a growing ecosystem of MCP servers for services ranging from databases and file systems to cloud APIs and developer tools.

Red Hat OpenShift AI (RHOAI) is an enterprise machine learning platform built on top of Kubernetes and Red Hat OpenShift. One of its core features is *workbenches* — containerised JupyterLab environments that data scientists and engineers use for model development and experimentation. Managing workbenches today requires navigating the RHOAI web UI or writing Kubernetes manifests directly, tasks that are repetitive, context-switching, and ill-suited to the natural-language interaction style that AI agents enable. No MCP server exists for RHOAI workbench management.

This thesis presents the design and implementation of an MCP server for RHOAI workbenches, written in Go using the official MCP Go SDK [2]. The server exposes a comprehensive set of tools covering the full lifecycle of workbench management: listing, creating, updating, and deleting workbenches; managing notebook images and hardware profiles; handling persistent volume claims; querying namespaces and pods; and monitoring resource consumption at the workbench,

namespace, user, and cluster level. A safety-oriented read-only mode is provided by default, with write operations gated behind an explicit environment variable. The server also exposes MCP resources and a guided prompt to assist agents in constructing correct workbench creation requests. Quality is validated through unit tests and automated evaluation using the Promptfoo framework [4], which measures tool selection accuracy, parameter extraction correctness, and execution success rate.

The remainder of this thesis is structured as follows. Chapter 2 introduces the background concepts: large language models, the shift toward AI agents and tool use, and the Model Context Protocol in detail. Chapter 3 covers Red Hat OpenShift AI, the Kubernetes resources underlying workbenches, and the motivation for MCP tooling in this context. Chapter 4 presents the analysis and design of the server, including requirements, architecture, and technology choices. Chapter 5 describes the implementation in full. Chapter 6 concludes the thesis and outlines directions for future work.

2 Background: Artificial Intelligence and the Model Context Protocol

2.1 Large Language Models and AI Assistants

At the core of modern AI assistants are *large language models* (LLMs) — neural networks trained to predict and generate text by learning statistical patterns from vast amounts of data. The dominant architectural foundation for LLMs is the *Transformer*, introduced by Vaswani et al. in 2017 [5]. The Transformer's self-attention mechanism allows the model to relate every token in a sequence to every other token, enabling it to capture long-range dependencies in text far more effectively than earlier recurrent architectures.

Modern LLMs are trained in two main stages. In the first stage, *pre-training*, the model is exposed to hundreds of billions of tokens of text and learns to predict the next token in a sequence. This process, while deceptively simple, causes the model to internalise broad world knowledge, reasoning patterns, and linguistic structure. In the second stage, the pre-trained model is fine-tuned to behave as a helpful assistant, typically through a combination of supervised instruction fine-tuning and reinforcement learning from human feedback (RLHF) [3]. The result is a model that responds to natural-language instructions in a coherent and useful way.

The products of this pipeline — systems such as OpenAI's ChatGPT, Anthropic's Claude, and Google's Gemini — are commonly referred to as *AI assistants*. An AI assistant takes a user's message as input and produces a textual response. It is fundamentally *stateless* and *passive*: it answers questions, drafts documents, and explains concepts, but it does not take actions in the world, does not maintain persistent memory across sessions by default, and cannot access information beyond its training data or the content of the current conversation. This passivity is the key limitation that the agentic paradigm, described in the next section, seeks to overcome.

2.2 AI Agents and Tool Use

Despite their impressive capabilities, pure language models have a fundamental constraint: they are isolated from the external world. Their knowledge has a training cutoff date, they cannot query live systems, and they cannot take actions with real-world consequences. The concept of an *AI agent* addresses this by equipping a language model with *tools* — discrete, callable functions that the model can invoke to interact with external systems, retrieve fresh information, or modify state.

The key mechanism enabling agents is *tool calling* (also referred to as *function calling*). When a model is given a set of tool definitions alongside a user request, it can — instead of producing a plain text response — output a structured call to one of those tools, specifying the tool name and the required arguments. The host application intercepts this call, executes the corresponding function, and feeds the result back into the model's context, after which the model continues its reasoning. This loop can repeat multiple times within a single interaction, allowing the model to chain tool calls to complete complex, multi-step tasks.

A widely cited formalisation of this pattern is the *ReAct* framework [6], which interleaves *reasoning* traces with *acting* steps. At each step, the model first produces a chain-of-thought reasoning trace — thinking through what it knows and what it needs — and then selects and executes an action (a tool call). The observation returned by the tool becomes part of the context for the next reasoning step. This structured loop gives agents a degree of *autonomy*: given a high-level goal, an agent can decompose it into sub-tasks, select appropriate tools, and adapt its plan based on the results it receives, without requiring the user to specify each individual step.

The practical impact of tool use is significant. An agent equipped with the right tools can list resources in a Kubernetes cluster, start or stop workbenches, query resource consumption metrics, and create new infrastructure — all in response to a single natural-language instruction such as "show me which workbenches are running in the production namespace and stop any that have been idle for more than a day". Realising this kind of interaction for Red Hat OpenShift AI workbenches requires a standardised way to expose those capabilities

to any AI agent — which is precisely the problem the Model Context Protocol solves.

2.3 The Model Context Protocol

2.3.1 Protocol Overview

2.3.2 MCP Clients and Servers

2.3.3 Transport Mechanisms

2.3.4 Tools, Resources, and Prompts

3 Background: Red Hat OpenShift AI and Workbenches

3.1 Kubernetes and OpenShift

3.2 Red Hat OpenShift AI

3.3 Workbenches

3.4 MCP Tooling in RHOAI

4 Analysis and Design

4.1 Motivation and Goals

4.2 Requirements

4.3 Architecture

4.4 Technology Choices

5 Implementation

5.1 Project Structure

5.2 MCP Tools

5.2.1 Workbench Management

5.2.2 Hardware Profiles

5.2.3 Notebook Images

5.2.4 Storage and Namespaces

5.2.5 Resource Consumption

5.3 MCP Resources and Prompts

5.4 Security

5.5 Code Quality

5.6 Evaluation

6 Conclusion

6.1 Future Work: Notebooks 2.0

Bibliography

[1] Anthropic. *Introducing the Model Context Protocol*. Nov. 25, 2024. URL: https://www.anthropic.com/news/model-context-protocol (visited on 03/10/2026).

[2] Model Context Protocol Contributors. *Go SDK for Model Context Protocol*. 2025. URL: https://github.com/modelcontextprotocol/go-sdk (visited on 03/10/2026).

[3] Long Ouyang et al. "Training language models to follow instructions with human feedback". In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 27730–27744. URL: https://arxiv.org/abs/2203.02155.

[4] Promptfoo Contributors. *Promptfoo: Test and evaluate LLMs*. 2024. URL: https://www.promptfoo.dev (visited on 03/10/2026).

[5] Ashish Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems* 30 (2017). URL: https://arxiv.org/abs/1706.03762.

[6] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *International Conference on Learning Representations*. 2023. URL: https://arxiv.org/abs/2210.03629.